

Name: Anvith Shetty

PRN: 202401100138

BATCH : A4

AIML MDM LAB

Practical Assignment 1.2: Expert System

Expert System for Plant Disease Diagnosis using Prolog

Problem Statement (PS):

To develop a rule-based expert system in Prolog that diagnoses plant diseases based on observed symptoms (such as brown spots, wilting, yellow leaves, white powder, stunted growth, and leaf curling) and recommends an appropriate remedy, by encoding domain knowledge as facts and inference rules in a knowledge base.

CODE:

```
:- dynamic patient_symptom/1.
```

```
:- dynamic diagnosed/1.
```

```
symptom(yellow_leaves).
```

```
symptom(brown_spots).
```

```
symptom(wilting).
```

```
symptom(white_powder).
```

```
symptom(stunted_growth).
```

```
symptom(leaf_curling).
```

```
disease_symptoms(leaf_blight,      [brown_spots, wilting]).  
disease_symptoms(powdery_mildew,   [white_powder, yellow_leaves]).  
disease_symptoms(root_rot,         [wilting, yellow_leaves,  
stunted_growth]).  
disease_symptoms(mosaic_virus,     [leaf_curling, stunted_growth,  
yellow_leaves]).
```

```
remedy(leaf_blight,      'Apply copper-based fungicide and remove infected  
leaves.').
```

```
remedy(powdery_mildew, 'Apply sulfur spray and improve air  
circulation.').
```

```
remedy(root_rot,        'Improve soil drainage and reduce watering  
frequency.').
```

```
remedy(mosaic_virus,    'Remove infected plants and control aphid  
population.').
```

```
patient_symptom(brown_spots).
```

```
patient_symptom(wilting).
```

```
match_count(Disease, Count) :-
```

```
    disease_symptoms(Disease, Symptoms),  
    findall(S, (member(S, Symptoms), patient_symptom(S)), Matched),  
    length(Matched, Count).
```

```
candidate(Disease, Count) :-
```

```
    disease_symptoms(Disease, _),  
    match_count(Disease, Count),
```

Count > 0.

diagnose(BestDisease) :-

```
    findall(Count-Disease, candidate(Disease, Count), Pairs),  
    Pairs \= [],  
    keysort(Pairs, Sorted),  
    last(Sorted, _-BestDisease).
```

perform(diagnose) :-

```
    diagnose(Disease), !,  
    remedy(Disease, Remedy),  
    format("Diagnosis: Plant likely has ~w~n", [Disease]),  
    format("Recommended remedy: ~w~n", [Remedy]),  
    assert(diagnosed(Disease)).
```

perform(no_match) :-

```
    format("No matching disease found for given symptoms. Consult an  
expert.~n", []).
```

start :-

```
( diagnose(_) -> perform(diagnose) ; perform(no_match) ).
```

show_symptoms :-

```
    findall(S, patient_symptom(S), List),  
    format("Current recorded symptoms: ~w~n", [List]).
```

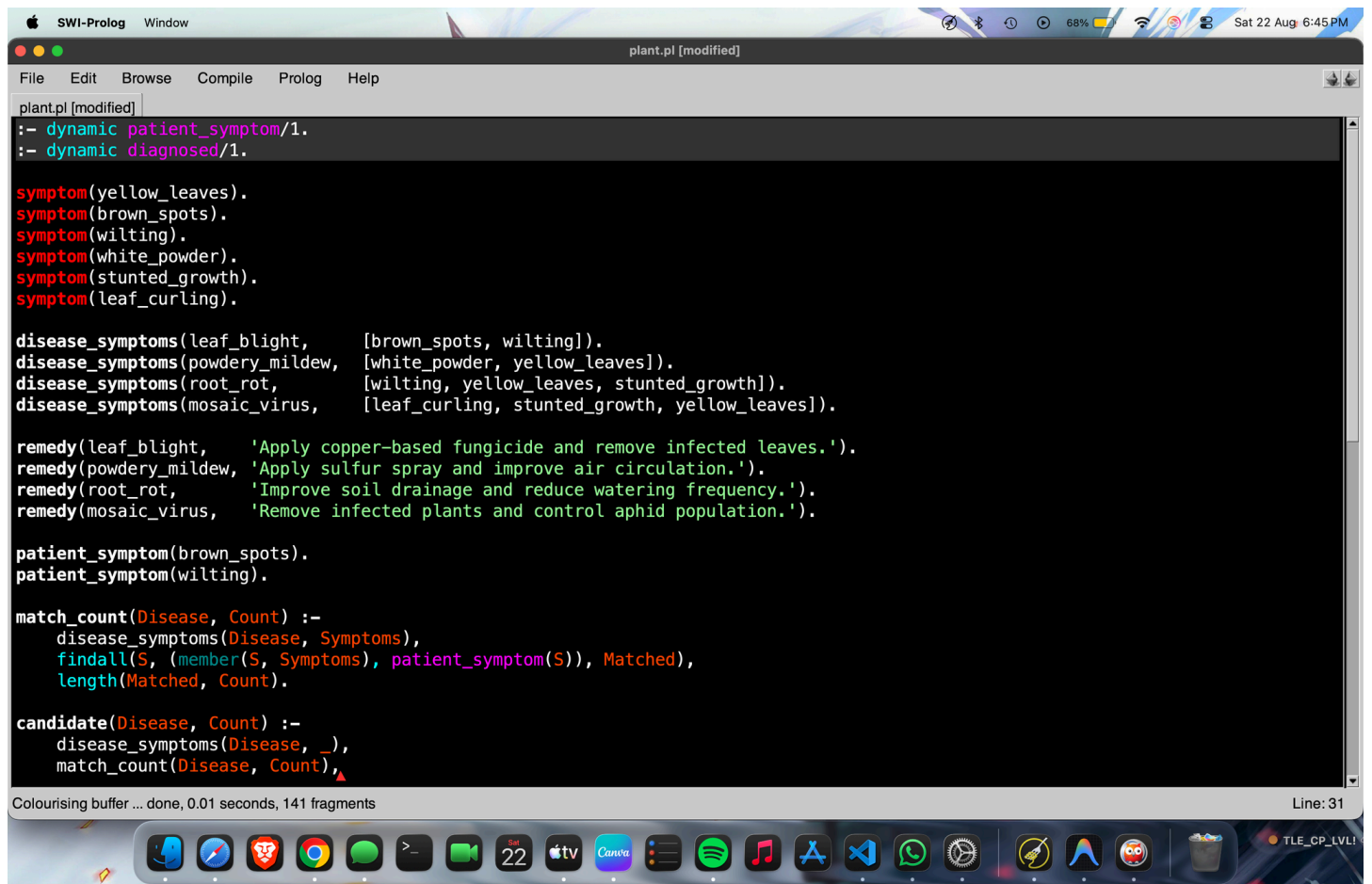
add_symptom(S) :-

```
( patient_symptom(S) -> true ; assert(patient_symptom(S)) ),  
format("Symptom '~w' recorded.~n", [S]).
```

reset :-

```
retractall(patient_symptom(_)),  
retractall(diagnosed(_)),  
assert(patient_symptom(brown_spots)),  
assert(patient_symptom(wilting)),  
format("Simulation reset. Default symptoms loaded.~n", []).
```

OUTPUT:



A screenshot of a Prolog IDE window titled "SWI-Prolog Window" with a file named "plant.pl [modified]". The menu bar includes "File", "Edit", "Browse", "Compile", "Prolog", and "Help". The code in the editor is as follows:

```
plant.pl [modified]
:- dynamic patient_symptom/1.
:- dynamic diagnosed/1.

symptom(yellow_leaves).
symptom(brown_spots).
symptom(wilting).
symptom(white_powder).
symptom(stunted_growth).
symptom(leaf_curling).

disease_symptoms(leaf_blight, [brown_spots, wilting]).
disease_symptoms(powdery_mildew, [white_powder, yellow_leaves]).
disease_symptoms(root_rot, [wilting, yellow_leaves, stunted_growth]).
disease_symptoms(mosaic_virus, [leaf_curling, stunted_growth, yellow_leaves]).

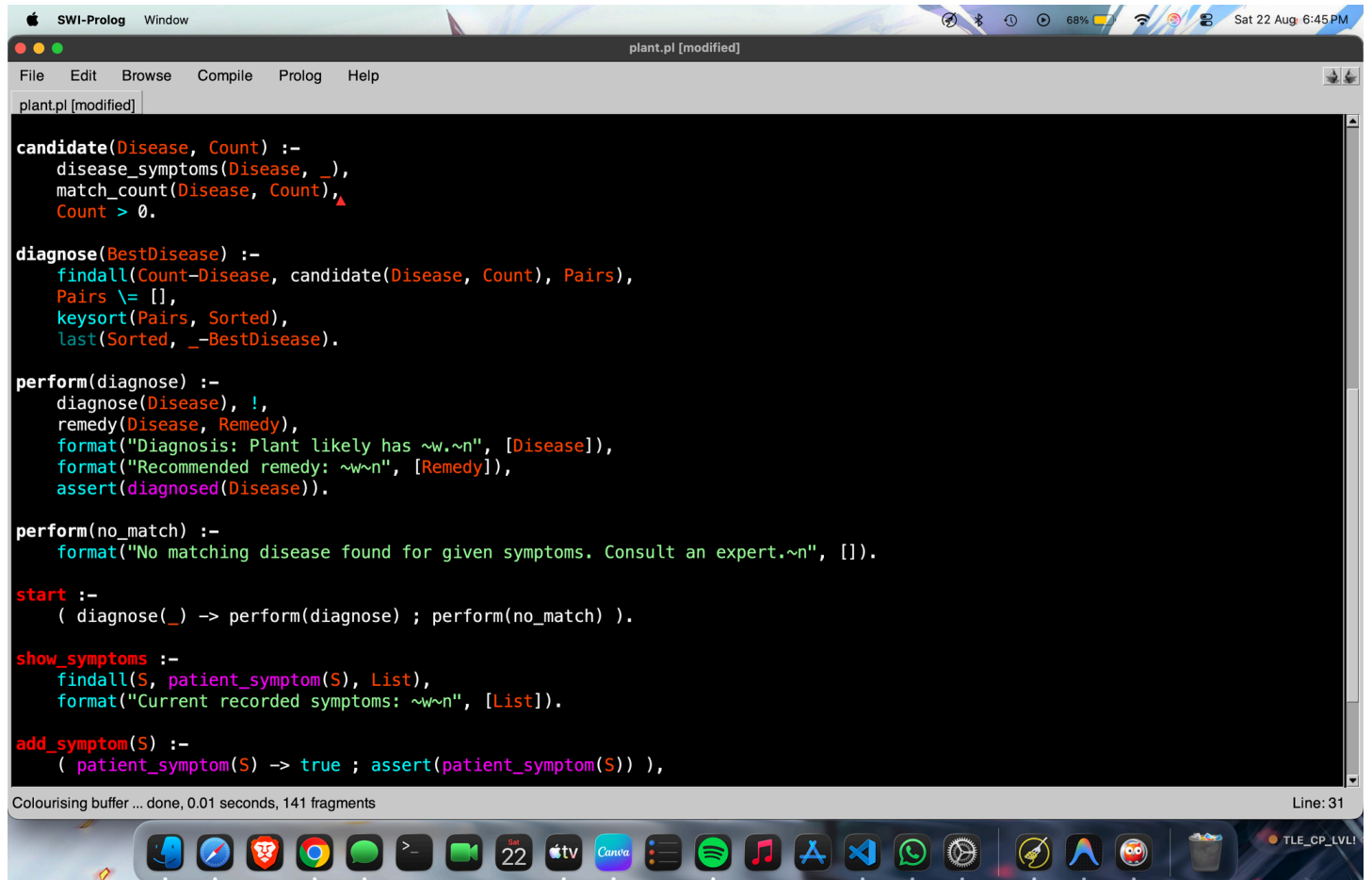
remedy(leaf_blight, 'Apply copper-based fungicide and remove infected leaves.').
remedy(powdery_mildew, 'Apply sulfur spray and improve air circulation.').
remedy(root_rot, 'Improve soil drainage and reduce watering frequency.').
remedy(mosaic_virus, 'Remove infected plants and control aphid population.').

patient_symptom(brown_spots).
patient_symptom(wilting).

match_count(Disease, Count) :-
    disease_symptoms(Disease, Symptoms),
    findall(S, (member(S, Symptoms), patient_symptom(S)), Matched),
    length(Matched, Count).

candidate(Disease, Count) :-
    disease_symptoms(Disease, _),
    match_count(Disease, Count),
```

The status bar at the bottom indicates "Colourising buffer ... done, 0.01 seconds, 141 fragments" and "Line: 31". The macOS dock is visible at the bottom with various application icons.



A screenshot of the same Prolog IDE window, showing the continuation of the "plant.pl" code:

```
Count > 0.

diagnose(BestDisease) :-
    findall(Count-Disease, candidate(Disease, Count), Pairs),
    Pairs \= [],
    keysort(Pairs, Sorted),
    last(Sorted, _-BestDisease).

perform(diagnose) :-
    diagnose(Disease), !,
    remedy(Disease, Remedy),
    format("Diagnosis: Plant likely has ~w~n", [Disease]),
    format("Recommended remedy: ~w~n", [Remedy]),
    assert(diagnosed(Disease)).

perform(no_match) :-
    format("No matching disease found for given symptoms. Consult an expert.~n", []).

start :-
    ( diagnose(_) -> perform(diagnose) ; perform(no_match) ).

show_symptoms :-
    findall(S, patient_symptom(S), List),
    format("Current recorded symptoms: ~w~n", [List]).

add_symptom(S) :-
    ( patient_symptom(S) -> true ; assert(patient_symptom(S)) ),
```

The status bar at the bottom shows "Colourising buffer ... done, 0.01 seconds, 141 fragments" and "Line: 31". The macOS dock is visible at the bottom.

```
SWI-Prolog Window
plant.pl [modified]
File Edit Browse Compile Prolog Help
plant.pl [modified]
diagnose(Disease), !,
remedy(Disease, Remedy),
format("Diagnosis: Plant likely has ~w.~n", [Disease]),
format("Recommended remedy: ~w~n", [Remedy]),
assert(diagnosed(Disease)).

perform(no_match) :-
    format("No matching disease found for given symptoms. Consult an expert.~n", []).

start :-
    ( diagnose(_) -> perform(diagnose) ; perform(no_match) ).

show_symptoms :-
    findall(S, patient_symptom(S), List),
    format("Current recorded symptoms: ~w~n", [List]).

^add_symptom(S) :-
    ( patient_symptom(S) -> true ; assert(patient_symptom(S)) ),
    format("Symptom '~w' recorded.~n", [S]).

reset :-
    retractall(patient_symptom(_)),
    retractall(diagnosed(_)),
    assert(patient_symptom(brown_spots)),
    assert(patient_symptom(wilting)),
    format("Simulation reset. Default symptoms loaded.~n", []).

Line: 56
```

```
SWI-Prolog Window
SWI-Prolog console
File Settings Tools Debug Help
37 ?- consult(['/Users/anwith_007/Prolog/plants.pl']).
true.

38 ?- reset.
Simulation reset. Default symptoms loaded.
true.

39 ?- start.
Diagnosis: Plant likely has leaf_blight.
Recommended remedy: Apply copper-based fungicide and remove infected leaves.
true.

40 ?- retractall(patient_symptom(_)).
true.

41 ?- add_symptom(white_powder).
Symptom 'white_powder' recorded.
true.

42 ?- add_symptom(yellow_leaves).
Symptom 'yellow_leaves' recorded.
true.

43 ?- start.
Diagnosis: Plant likely has powdery_mildew.
Recommended remedy: Apply sulfur spray and improve air circulation.
true.

44 ?- ^
```

Result:

Ran the expert system with two different symptom sets to check if the diagnosis logic actually works. First with the default symptoms (brown spots & wilting) so it correctly matched leaf blight and gave the fungicide remedy. Then changed the symptoms to white powder + yellow leaves and reran it — this time it correctly switched the diagnosis to powdery mildew with the sulfur spray remedy. So the rules are working as expected, the system isn't just giving one fixed answer, it actually reasons based on whatever symptoms are given.